ПРИЛОЖЕНИЕ 1

Техническое задание

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

Кафедра «Системы обработки информации и управления»

СОГЛАСОВАНО	УТВЕРЖДАЮ
Руководитель	Заведующий кафедрой ИУ-5 (Индекс кафедры)
_____ А.И. Канев _____ (подпись) (инициалы и фамилия)	_____ В.И. Терехов (подпись) (инициалы и фамилия)
«____» _____ 2026 г.	«____» _____ 2026 г.

Определение объёма земляных работ при разработке котлована

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

(вид документа)

по ГОСТ 19.201-78 Единая система программной документации. Техническое задание. Требования к содержанию и оформлению

писчая бумага
(вид носителя данных)

Листов 9

Исполнитель: студент ИУ5–54Б

_____ К.И. Давыдов
(подпись) (инициалы и фамилия)

«____» _____2026 г.

2026

1 Цель

Разработать веб-систему ведения и просмотра справочника типов грунта с коэффициентами разрыхления и плотностью для подготовки исходных данных при определении объёма земляных работ при разработке котлована.

2 Назначение

Система предназначена для инженера, выбирающего характеристики грунта, и создателя карточек, ведущего справочник. Гость просматривает опубликованные типы грунта в каталоге и ленте; создатель после входа добавляет и публикует записи, удаляет свои карточки и ставит лайки. В рамках текущего этапа расчёт объёма котлована не реализуется: приложение обеспечивает справочные данные по выбранной теме.

3 Задачи разработки

3.1 Разработать мобильные макеты ленты, каталога и добавления типа грунта в Figma; реализовать серверные страницы на NestJS, TypeScript и Handlebars, подключить статические файлы и MinIO в Docker (ЛР1).

3.2 Спроектировать таблицы soil_types, users и likes в PostgreSQL; реализовать модели и миграции TypeORM, шаблоны и операции с данными на NestJS; проверить БД в Adminer (ЛР2).

3.3 Разработать REST API из десяти методов в двух доменах NestJS, загрузку фото и видео в MinIO, фильтрацию, публикацию, лайки и логическое удаление; проверить запросы в Postman (ЛР3).

3.4 Реализовать регистрацию и аутентификацию с bcrypt, cookie и Redis на NestJS с TypeORM; разграничить доступ и описать API средствами Swagger; подготовить ТЗ в Word (ЛР4).

3.5 Разработать базовое SPA для гостя: каталог и ленту типов грунта, фильтр по коэффициенту разрыхления, mock-данные и медиа по умолчанию средствами React, TypeScript, React-Bootstrap и Vite (ЛР5).

3.6 Добавить вход и выход, создание, публикацию, удаление и лайки на React; подключить API через Axios, кодогенерацию из Swagger, Redux Toolkit и redux-thunk; использовать proxy и гостевой mock при недоступности API (ЛР6).

3.7 Сохранять фильтр в Redux Toolkit, реализовать адаптивность CSS для компьютера и телефона, PWA с manifest и Service Worker; развернуть собранное приложение на веб-сервере (ЛР7).

3.8 Разместить демонстрационный фронтенд с mock-данными на GitHub Pages; проверить переходы, фильтрацию и установку PWA на телефоне (Vite, GitHub Pages, HTTPS).

3.9 Подготовить РПЗ, ТЗ, таблицу методов, отчёты по исправлениям и скриншоты в Word; поддерживать ER, классы и последовательность в одном файле StarUML; проверить оформление средствами просмотра PDF и TestVKR.

3.10 Вести отдельные репозитории бэкенда и фронтенда в Git/GitHub, выделять каждую лабораторную работу в именованную ветку, описывать запуск, методы, таблицы и ссылки в README.

4 Функциональные требования

4.1 HTTP методы

4.1.1 GET /api/soil_types — Получение опубликованных типов грунта с верхней границей коэффициента разрыхления; доступно гостю и создателю.

4.1.2 GET /api/soil_types/feed — Получение одной опубликованной карточки: первой без id, указанной по id, следующей по кругу при next=true; доступно гостю и создателю.

4.1.3 GET /api/soil_types/draft — Получение своего единственного черновика без id; доступно создателю.

4.1.4 POST /api/soil_types — Создание своего черновика с названием, фото и коротким видео; доступно создателю.

4.1.5 PUT /api/soil_types/{id}/publish — Заполнение описания и характеристик, публикация своего черновика; доступно создателю.

4.1.6 DELETE /api/soil_types/{id} — Логическое удаление своей карточки; доступно создателю.

4.1.7 POST /api/soil_types/{id}/like — Установка или отмена своего лайка опубликованному грунту; доступно создателю.

4.1.8 POST /api/users/register — Регистрация гостя с уникальным логином и паролем; роль модератора клиентом не задаётся.

4.1.9 POST /api/users/login — Аутентификация гостя по логину и паролю; создание сессии Redis и установка cookie sessionId.

4.1.10 POST /api/users/logout — Деавторизация: удаление сессии Redis и очистка cookie; повторный выход без сессии допустим.

4.2 Меню

4.2.1 Меню отображается снизу на четырёх страницах и доступно гостю и создателю.

4.2.2 «Лента» открывает страницу 4.5, «Каталог» — страницу 4.4. Оба пункта доступны всем.

4.2.3 «Добавление» открывает страницу 4.6; доступно создателю. Для гостя пункт неактивен.

4.2.4 «Вход» открывает страницу 4.3 для гостя; после входа меняется на «Выход». Выход вызывает метод 4.1.10, сбрасывает фильтр и переключает интерфейс в состояние гостя.

4.3 Вход в каталог грунтов

4.3.1 Страница доступна гостю и содержит поля login и password, кнопки «Войти» и «Зарегистрироваться». Поле пароля маскируется.

4.3.2 «Войти» вызывает метод 4.1.9. При успехе открывается каталог в интерфейсе создателя. Ошибка аутентификации выводится рядом с формой.

4.3.3 «Зарегистрироваться» вызывает метод 4.1.8. Логин обязателен, длина до 50 символов; пароль — от 4 до 100 символов. Повторный логин отклоняется; пароль и его хеш не выводятся в ответе.

4.3.4 Сессия сохраняется в HttpOnly cookie, срок жизни — один час. После обновления React-страницы клиентское состояние входа сбрасывается в гостевое, cookie остаётся в браузере до истечения срока либо выхода.

4.4 Каталог типов грунта

4.4.1 Страница доступна гостю и создателю. Метод 4.1.1 возвращает карточки: изображение, название, коэффициент разрыхления, плотность в т/м³ и количество лайков. При отсутствии изображения используется локальная заглушка.

4.4.2 Поле или слайдер maxLoosening и кнопка с лупой вызывают метод 4.1.1 с максимальным коэффициентом разрыхления. Фильтрация выполняется в БД; значение сохраняется при переходе в ленту и обратно.

4.4.3 Нажатие на карточку открывает ленту 4.5 и вызывает метод 4.1.2 с id выбранного грунта.

4.4.4 «Удалить» доступно создателю на своей карточке при isMine=1; вызывает метод 4.1.6. Сервер независимо проверяет владельца; удалённая карточка исключается из списка.

4.5 Лента типов грунта

4.5.1 Страница доступна гостю и создателю, показывает одну карточку: вертикальное видео, название, описание с раскрытием «Ещё», коэффициент разрыхления, плотность и количество лайков. При отсутствии видео используется локальная заглушка.

4.5.2 При открытии без выбранного грунта вызывается метод 4.1.2 без id; при переходе из каталога — с id выбранной карточки.

4.5.3 Действие «Следующий» вызывает тот же метод 4.1.2 с текущим id и next=true. После последней карточки показывается первая опубликованная; пропуски идентификаторов допускаются.

4.5.4 Лайк доступен создателю: метод 4.1.7 с value=1 устанавливает, с value=0 снимает лайк. Ответ обновляет likesCount и isLiked. Повторная установка не создаёт второй лайк.

4.6 Добавление типа грунта

4.6.1 Страница доступна создателю. При открытии метод 4.1.3 получает его черновик. Если черновика нет, ответ 404 означает начало заполнения новой карточки.

4.6.2 Первый шаг содержит название name (обязательно, до 100 символов) и выбор необязательных файлов image и video; фото располагается над видео. «Далее» вызывает метод 4.1.4 с этими тремя полями multipart/form-data. Ответ содержит только id; повторный черновик запрещён.

4.6.3 После «Далее» файлы и название не меняются. Пользователь заполняет description (до 1000 символов), looseningFactor (1–2) и density (0,5–3 т/м³). Все три поля обязательны для публикации.

4.6.4 «Опубликовать» вызывает метод 4.1.5: id находится в URL, в JSON передаются ровно description, looseningFactor и density. Сервер устанавливает статус published и дату публикации. После успеха открывается опубликованная карточка.

4.6.5 Пользователь определяется по сессии; status, creatorId, createdAt и formedAt не принимаются из формы. Неизвестные поля дают 400, отсутствие входа — 401; чужой черновик не публикуется.

5 Требования к аппаратному обеспечению

5.1 Расчёт аппаратных требований

Проектные допущения: 20 создателей, одна новая карточка на создателя в день; 8 HTTP-запросов на цикл публикации по диаграмме; 1000 гостей по 20 запросов за рабочие 8 часов. Оценка одного ядра — 500 запросов/с, запас — 1,5; это не результат нагрузочного испытания.

$$RPS = \frac{20 \times 1 \times 8 + 1000 \times 20}{8 \times 3600} = 0{,}7 \ \textit{запросов/с}$$

$$\textit{Ядра} = \lceil 0{,}7 \times 1{,}5 \ / \ 500 \rceil = 1; \quad \textit{ОП} = 1 \times 4 = 4 \ \textit{ГБ}$$

Бюджет полей soil_types: для UTF-8 принято до 4 байт/символ; NUMERIC — до 8 байт в выбранных диапазонах. Заголовки строк, индексы и выравнивание учтены коэффициентом 2 в общем объёме.

Поле	Тип	Байт
id	INTEGER	4
name	VARCHAR(100)	400
description	VARCHAR(1000)	4000
status	VARCHAR(20)	80
imageUrl	VARCHAR(255)	1020
videoUrl	VARCHAR(255)	1020
looseningFactor	NUMERIC(4,2)	8
density	NUMERIC(4,2)	8
createdAt	TIMESTAMP	8
formedAt	TIMESTAMP	8
creatorId	INTEGER	4
Всего на карточку		6560

likes: id + userId + soilTypeId = 4 + 4 + 4 = 12 байт; принято 3 лайка на карточку. users: id 4 + login 200 + password (ASCII-хеш) 100 + isModerator 1 = 305 байт; 20 записей. Гости не регистрируются. Удалённые карточки сохраняются.

$$\textit{Карточек за 5 лет} = 20 \times 1 \times 365 \times 5 = 36\ 500$$

$$\textit{БД} = \frac{2 \times (36\ 500 \times (6560 + 3 \times 12) + 20 \times 305)}{1\ 073\ 741\ 824} \approx 0{,}448 \ \textit{ГиБ}$$

$$\textit{Медиа} = 36\ 500 \times (1 + 5) \ / \ 1024 \approx 213{,}87 \ \textit{ГиБ}$$

Принято среднее фото 1 МиБ и видео 5 МиБ. Вместе с 30 ГиБ для ОС и контейнеров требуется около 244,32 ГиБ (262,33 ГБ). Выбирается SSD 500 ГБ; резервные копии размещаются на отдельном носителе. Для совместного запуска NestJS, PostgreSQL, Redis и MinIO принимаются 2 ядра и 8 ГБ ОП (4 ГБ на ядро).

5.2 Сервер

Для рассчитанной нагрузки необходим сервер с двумя процессорными ядрами, 8 ГБ оперативной памяти и SSD не менее 500 ГБ; подключение к сети — от 100 Мбит/с. Учебный стенд может использовать 40 ГБ свободного места с расширением по мере накопления медиа. Резервное копирование PostgreSQL и объектов MinIO выполняется на отдельный носитель.

5.3 Клиент

Компьютер с двухъядерным процессором, 4 ГБ ОП, клавиатурой и мышью либо смартфон с сенсорным экраном; ширина интерфейса от 360 CSS-пикселей. Подключение к Интернету от 10 Мбит/с для загрузки короткого видео. Сервер и клиент могут работать на разных устройствах.

6 Требования к программному обеспечению

6.1 Сервер

Проверенный учебный стенд: macOS 26.6, Docker Engine 29.2.0, Compose 5.0.2; приложение выполняется на Node.js 24.19.0. Контейнер PostgreSQL 12.22 хранит данные; Redis 6.2.24 — сессии; MinIO RELEASE.2025-09-07T16-13-09Z — файлы фото и видео.

Версии из package-lock.json: NestJS 11.2.3, TypeScript 5.9.3, TypeORM 1.1.1, @nestjs/swagger 11.4.7, Handlebars (hbs) 4.3.0, bcryptjs 3.0.3, ioredis 5.11.1 и pg 8.23.0. TypeORM сохраняется по указанию преподавателя; Sequelize не используется. Миграции обязательны, synchronize=false. Для развёртывания допускается Linux с Docker и совместимой версией Node.js 24.x.

Сессии хранятся в Redis с TTL 3600 секунд; cookie sessionId содержит случайный идентификатор и флаги HttpOnly, SameSite=Lax, Path=/. В рабочем окружении необходим HTTPS и Secure. Секреты задаются переменными окружения; файл .env исключается из Git. Swagger UI и схема OpenAPI доступны по /api и /api-json.

6.2 Клиент

Требуется браузер с поддержкой HTML5, CSS Grid/Flexbox, JavaScript ES2022, cookie и воспроизведения MP4. Для целевого фронтенда следующих этапов устанавливаются проектные диапазоны: React 19.x, TypeScript 5.x, React-Bootstrap 2.x, Axios 1.x, Redux Toolkit 2.x, Vite 7.x. Это требования к будущей сборке, а не перечень уже установленных пакетов. Точные версии фиксируются в lock-файле фронтенда; React Context не применяется.

PWA использует manifest и Service Worker и работает через HTTPS; демонстрационная версия размещается на GitHub Pages. Для подготовки и проверки материалов требуются Git, Postman, StarUML и редактор с поддержкой DOCX/DOC. Исходники, HTTP-коллекции и документация должны соответствовать одной версии API.